

Московский Авиационный Институт
(Национальный Исследовательский Университет)
Факультет информационных технологий и прикладной математики
Кафедра вычислительной математики и программирования

**Лабораторная работа №2 по курсу
«Операционные системы»**

Студент: Черноус Алексей Тимофеевич
Группа: М8О-209Б-23
Вариант: 7
Преподаватель: Миронов Евгений Сергеевич
Оценка: _____
Дата: _____
Подпись: _____

Москва, 2024

Содержание

1. Репозиторий
2. Постановка задачи
3. Общие сведения о программе
4. Общий метод и алгоритм решения
5. Исходный код
6. Демонстрация работы программы
7. Выводы

Репозиторий

https://github.com/HGRaicer/MAI_OS/tree/main/lab2

Постановка задачи

Цель работы

Приобретение практических навыков в:

- Управление потоками в ОС
- Обеспечение синхронизации между потоками

Задание

Составить программу на языке Си, обрабатывающую данные в многопоточном режиме. При обработке использовать стандартные средства создания потоков операционной системы (Windows/Unix). Ограничение потоков должно быть задано ключом запуска программы.

Необходимо уметь продемонстрировать количество потоков, используемых программой, с помощью стандартных средств операционной системы. Привести исследование зависимости ускорения и эффективности алгоритма от входящих данных и количества потоков. Объяснить получившиеся результаты.

Вариант 7: Два человека играют в кости. Правила игры следующие: каждый игрок делает бросок 2-ух

костей К раз; побеждает тот, кто выбросил суммарно большее количество очков. Задача

программы экспериментально определить шансы на победу каждого из игроков. На вход программе подается К, какой сейчас тур, сколько очков суммарно у каждого из игроков и количество экспериментов, которые должна произвести программа

Общие сведения о программе

Программа написана на языке Си в UNIX-подобной операционной системе.

Для компиляции программы требуется указать ключ -pthread. Для запуска программы в качестве 1 аргумента командной строки необходимо указать максимальное количество потоков, 2 аргумента – количество бросков кубиков, 3 аргумент – количество очков у первого игрока, 4 аргумент – количество очков у второго игрока, 5 аргумент – количество экспериментов.

Общий метод и алгоритм решения

Пусть на вход подается n потоков и m количества экспериментов. Тогда

каждый поток будет обрабатывать m/n экспериментов. Каждый поток запускает функцию обрабатывающую одну игру, где проверяется не превысило ли количество экспериментов ограничение, а затем запускается цикл симулирующий k бросков кубиков. По окончании цикла программа смотрит у кого игрока больше очков и тому засчитывает победу. В итоге, после отработки всех потоков имеется 3 переменные отвечающие за количество побед первого игрока, количество побед второго игрока и количество ничей.

Исходный код

```
main.cpp
#include <pthread.h>
#include <stdlib.h>
#include <stdio.h>
#include <time.h>
#include <semaphore.h>
#include <stdbool.h>

typedef struct {
    unsigned int k;
    int* res;
    pthread_mutex_t* mutex;

    const int score1;
    const int score2;

    int expr;
}Data;
```

```
int occ = 0;
```

```
void *simul_game(void* arg){  
    printf("Thread ID: %lu\n", pthread_self());  
    Data* data = (Data*)arg;
```

```
    while(true) {  
        pthread_mutex_lock(data->mutex);  
        int cell = occ++;  
        pthread_mutex_unlock(data->mutex);  
  
        if (cell >= data->expr) break;  
  
        int score1 = data->score1, score2 = data->score2;  
  
        for(unsigned int i=0; i < data->k; ++i){  
            score1 = rand() % 11 + 2;  
            score2 = rand() % 11 + 2;  
        }  
  
        if (score1 > score2){  
            pthread_mutex_lock(data->mutex);  
            data->res[0]++;  
            pthread_mutex_unlock(data->mutex);  
        }
```

```

else if (score2 > score1){
    pthread_mutex_lock(data->mutex);
    data->res[1]++;
    pthread_mutex_unlock(data->mutex);
} else {
    pthread_mutex_lock(data->mutex);
    data->res[2]++;
    pthread_mutex_unlock(data->mutex);
}
}
pthread_exit(0);
}

```

```

int main(int argc, char *argv[]){
    if (argc < 6){
        fprintf(stderr,"Usage %s <max threads> <number of thows> <score1>
<score2> <number of experiments>", argv[0]);
        return 1;
    }

```

```

    unsigned int k = atoi(argv[2]);
    int score1 = atoi(argv[3]), score2 = atoi(argv[4]);
    unsigned int max_threads = atoi(argv[1]);

```

```

unsigned int expr = atoi(argv[5]);

pthread_mutex_t mutex;
pthread_mutex_init(&mutex, NULL);

int res[3] = {0, 0, 0};
srand(time(NULL));

pthread_t* threads = malloc(max_threads * sizeof(pthread_t));

if (!threads){
    fprintf(stderr, "Failed to allocated memory");
    return 1;
}

Data data = {k, res, &mutex, score1, score2, expr};

int active_thrad = 0;

clock_t start = clock();
for (int i = 0; i < max_threads; ++i){
    if(pthread_create(&threads[i], NULL, simul_game, &data) != 0){
        fprintf(stderr, "Failed create thread");
        return 1;
    }
}
for (int i = 0; i < max_threads; ++i){
    pthread_join(threads[i], NULL);
}

```

```

    clock_t end = clock();
    double del_time = (double) end-start;
    printf("Player 1 wins: %d\n", res[0]);
    printf("Player 2 wins: %d\n", res[1]);
    printf("Draw: %d\n", res[2]);
    printf("Del time %f\n", del_time);

    free(threads);
    pthread_mutex_destroy(&mutex);

    return 0;
}

```

Демонстрация работы программы

```
rai@rai-laptop:~/cods/os/lab2$ ./a.out 4 1000000 0 0 16
```

```
Thread ID: 140363705214528
```

```
Thread ID: 140363680036416
```

```
Thread ID: 140363696821824
```

```
Thread ID: 140363688429120
```

```
Player 1 wins: 7
```

```
Player 2 wins: 8
```

```
Draw: 1
```

Выводы

Язык Си позволяет пользователю взаимодействовать с потоками операционной системы. Для этого на Unix-подобных системах требуется подключить библиотеку pthread.h.

Создание потоков происходит быстрее, чем создание процессов, а все потоки используют одну и ту же область данных. Поэтому многопоточность – один из способов ускорить обработку каких-либо данных: выполнение

однотипных, не зависящих друг от друга задач, можно поручить отдельным потокам, которые будут работать параллельно.

Средствами языка Си можно совершать системные запросы на создание потока, ожидания завершения потока, а также использовать различные примитивы синхронизации.